ΑΝΑΔΡΟΜΕΣ
Recursion

ΚΑΙ ΓΛΩΣΣΕΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ
ΑΝΑΔΡΟΜΕΣ & ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ C

Μ.Ι.Ε. 2021-2022

Η αναδρομή είναι μια τεχνική προγραμματισμού όπου ένας κώδικας καλεί τον ίδιο του τον εαυτό.

Με την επαναλαμβανόμενη αυτή διαδικασία ο ίδιος κώδικας μπορεί να εκτελείται πολλές φορές.

Η αναδρομή είναι μια τακτική που διαθέτουν κάποιες γλώσσες προγραμματισμού.

Εάν μια γλώσσα προγραμματισμού δεν περιλαμβάνει την δυνατότητα αναδρομής, κάθε προσπάθεια συγγραφής κώδικα με αναδρομή, οδηγεί σε απότομη διακοπή του προγράμματος.

Με την τακτική της αναδρομής ένα σύνθετο πρόβλημα μπορεί να διασπάται σε παρόμοια απλούστερα προβλήματα.

Συνήθως οι προγραμματιστές αποφεύγουν τις αναδρομές, καθώς είναι δύσκολες στην αντίληψη τους, αλλά υπάρχουν περιπτώσεις όπου μια αναδρομή μπορεί να είναι απαραίτητη καθώς μπορεί να δημιουργεί απλούστερο κώδικα.

Σε μια αναδρομή **η «μπίλια» που «τρέχει» μέσα στο κώδικα παγιδεύεται σε μια ατέρμονη διαδικασία**. Εάν η συγγραφή δεν περιλαμβάνει τα κατάλληλα σημεία διαφυγής (τέλος), τότε η αναδρομή, οδηγεί σε διακοπή του προγράμματος.

Μια αναδρομή μπορεί να αντικατασταθεί από μια κλασσική αλγοριθμική δομή επανάληψη (DO, WHILE, κ.λπ.). Πολλές φορές όμως η αναδρομή μπορεί να πετύχει καλύτερους χρόνους εκτέλεσης.

Υπάρχουν διαφόρων ειδών αναδρομές, άμεσες, έμμεσες, δενδροειδείς, αμοιβαίες, κ.λπ.

Μ.Ι.Σ. 2021-2022

Γενικότερα, κάθε κώδικας σε μια γλώσσα προγραμματισμού καταγράφει κατά την διάρκεια της εκτέλεσης του κάποιες απαραίτητες πληροφορίες (περιεχόμενο μεταβλητών, διεύθυνση μεταβλητών, διεύθυνση εισόδου σε μια ρουτίνα, διεύθυνση εξόδου από μια ρουτίνα, κ.λπ.).

Οι ανωτέρω περιγραφόμενες καταγραφές τοποθετούνται σε μια ειδική περιοχή της μνήμης που δημιουργούν οι γλώσσες προγραμματισμού που ονομάζεται στοίβα (stack).

Γίνεται αντιληπτό ότι στην στοίβα (stack), αποθηκεύονται όλες οι απαραίτητες πληροφορίες του κώδικα, σε διάταξη στοίβας, και καθώς ο κώδικας προχωρά στην εκτέλεση του, ανακτώνται οι κατάλληλες πληροφορίες μέσα από τον σωρό της στοίβας.

Η ίδια ανωτέρω διαδικασία εφαρμόζεται και στην τεχνική των αναδρομών, γίνεται εύκολα αντιληπτό ότι μια ατέρμονη αναδρομή ή μια αναδρομή με μεγάλο βάθος αναδρομικών επαναληπτικών κλήσεων η στοίβα θα υπερχειλίσει (overflow) και ο κώδικας οδηγεί σε απότομη διακοπή.

Μ.Ι.Σ. 2021-2022

ΠΑΡΑΔΕΙΓΜΑ ΚΩΔΙΚΑ ΜΕ ΑΝΑΔΡΟΜΗ ΓΙΑ ΤΗΝ ΕΥΡΕΣΗ ΤΗΣ ΑΚΟΛΟΥΘΙΑΣ ΦΙΜΠΟΝΑΤΣΙ

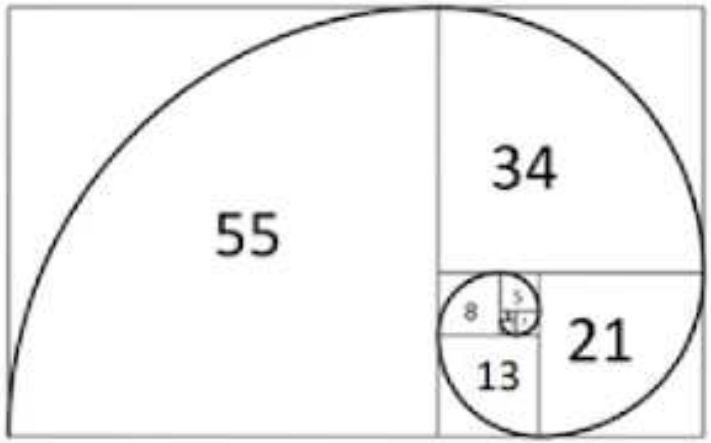
Να κάνετε πρόγραμμα σε C++ που θα δημιουργεί τους N πρώτους αριθμούς των αριθμων Fibonacci. Το N θα καθορίζεται από ερώτημα που θα θέτει το πρόγραμμα "Μέχρι ποιον αριθμό θέλετε την ακολουθία Fibonacci: "
Είναι δεδομένο ότι οι δύο πρώτοι αριθμοί είναι το 1 και το 1 και στην συνέχεια κάθε αριθμός προκύπτει από το άθροισμα των δύο προηγούμενων:

οι *Αριθμοί Φιμπονάτσι* είναι οι αριθμοί της παρακάτω ακέραιης ακολουθίας: **1,1,2,3,5,8,13,21,34,55,89,144,233,377…**

Εξ ορισμού, οι πρώτοι δύο αριθμοί Φιμπονάτσι είναι το 0 και το 1, **κάθε επόμενος αριθμός είναι το άθροισμα των δύο προηγούμενων.**

Η **Ακολουθία Φιμπονάτσι** ονομάστηκε από τον Λεονάρντο της Πίζας, γνωστό και ως Φιμπονάτσι

1+1=2	13+21=34
1+2=3	21+34=55
2+3=5	34+55=89
3+5=8	55+89=144
5+8=13	89+144=233
8+13=21	144+233=377



ΠΑΡΑΔΕΙΓΜΑ ΚΩΔΙΚΑ ΜΕ ΑΝΑΔΡΟΜΗ ΓΙΑ ΤΗΝ ΕΥΡΕΣΗ ΤΗΣ ΑΚΟΛΟΥΘΙΑΣ ΦΙΜΠΟΝΑΤΣΙ

```
#include <stdio.h>

int my_b_fibon(int n);
main()
{
    int x;
    while(1)
    {
        scanf("%d",&x);
        if (x<0)
            return;
        printf("O TELIKOS ARITMOS %d\n", my_b_fibon(x) );
    }
}

int my_b_fibon(int n)
{
    if (n==0)
            return 0;
    if (n==1)
            return 1;
    return ( my_b_fibon(n-1) + my_b_fibon(n-2) );
}
```

Μ.Ι.Σ. 2021-2022

The Fibonacci Sequence

1,1,2,3,5,8,13,21,34,55,89,144,233,377…

1 2 3 4 5 6 7 8 9 10 11 12 13 14

1+1=2
1+2=3
2+3=5
3+5=8
5+8=13
8+13=21

13+21=34
21+34=55
34+55=89
55+89=144
89+144=233
144+233=377

ΠΑΡΑΔΕΙΓΜΑ ΚΩΔΙΚΑ ΜΕ ΑΠΛΗ ΔΟΜΗ ΕΠΑΝΑΛΗΨΗΣ ΓΙΑ ΤΗΝ ΕΥΡΕΣΗ ΤΟΥ ΠΑΡΑΓΟΝΤΙΚΟΥ

```
#include <stdio.h>

int my_old_factorial(int n);

int main()

{
     int toses = 5;

     toses  =  my_old_factorial (toses);

    printf("TO APOTELESMA SAN ATROISMA ....%d", toses);
    scanf("DWSTE ENTER ...%d", &toses);

}

int my_old_factorial (int n)
{
   int i, my_result;
   my_result = 1;
   for (i = 2; i<= n; i++)
     {
            my_result = my_result * i;
     }
   return my_result;
}
```

Μ.Ι.Σ. 2021-2022

ΠΑΡΑΔΕΙΓΜΑ ΚΩΔΙΚΑ ΜΕ ΑΝΑΔΡΟΜΗ ΓΙΑ ΤΗΝ ΕΥΡΕΣΗ ΤΟΥ ΠΑΡΑΓΟΝΤΙΚΟΥ

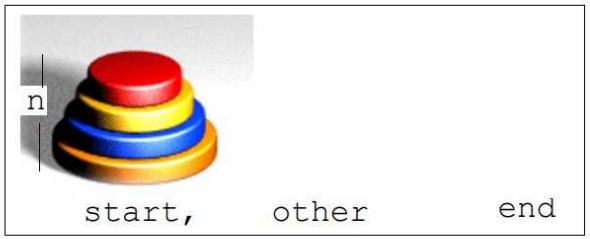
```
#include <stdio.h>

int my_b_factorial(int n);

int main()

{

     int toses = 5;

     toses  =  my_b_factorial (toses);

    printf("TO APOTELESMA SAN ATROISMA ....%d", toses);
    scanf("DWSTE ENTER ...%d", &toses);

}

int my_b_factorial(int n)
{
   if  (n == 0)
      return 1;

   return (n * my_b_factorial(n-1));

}
```

Μ.Ι.Σ. 2021-2022

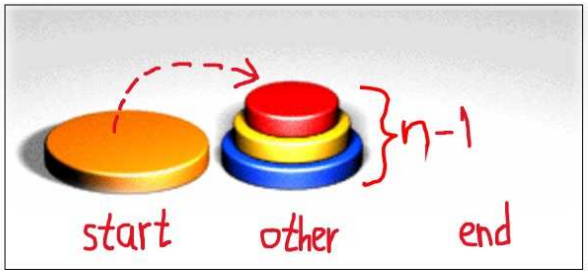
ΠΑΡΑΔΕΙΓΜΑ ΚΩΔΙΚΑ ΠΟΥ ΜΠΟΡΕΙ ΝΑ ΓΡΑΦΕΙ ΜΕ ΑΝΑΔΡΟΜΗ

ΓΙΑ ΤΗΝ ΕΠΑΝΑΤΟΠΟΘΕΤΗΣΗ ΤΟΥ ΠΥΡΓΟΥ ΤΟΥ ΑΝΟΙ ΣΕ ΕΝΑ ΔΙΠΛΑΝΟ ΝΗΣΑΚΙ (χωρίς να τοποθετήσουμε μεγαλύτερο τουβλάκι επάνω σε μικρότερο)

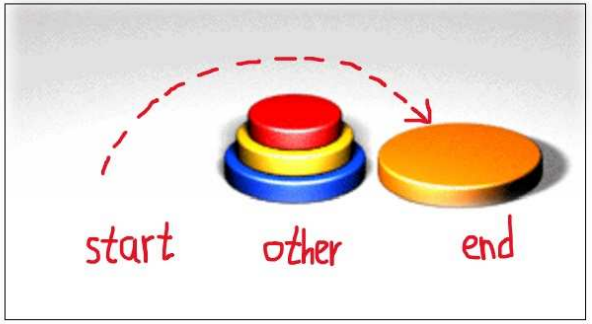
```
hanoi(n, start, end):
if n == 1:
    print(start, end)
```



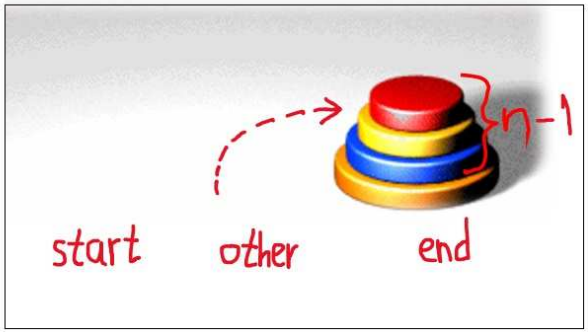

```
hanoi(n, start, end):
if n == 1:
    print(start, end)
else:
    other = 6 - start - end
    hanoi(n-1, start, other)
```

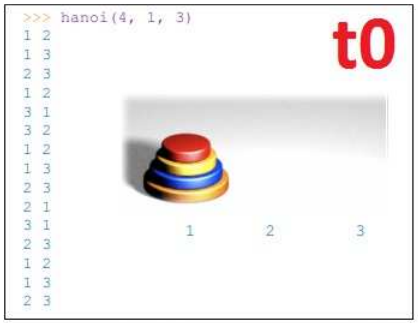


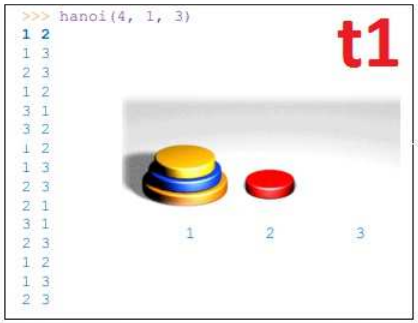

```
hanoi(n, start, end):
if n == 1:
    print(start, end)
else:
    other = 6 - start - end
    hanoi(n-1, start, other)
    hanoi(1, start, end)
```

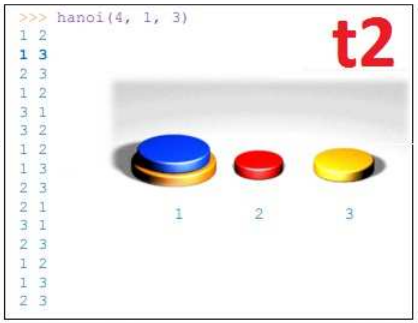


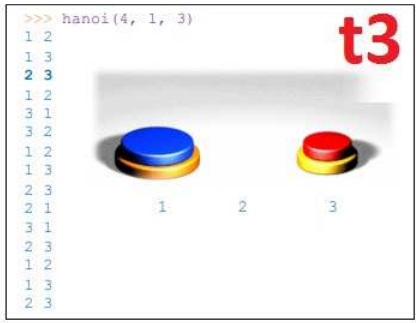

```
hanoi(n, start, end):
if n == 1:
    print(start, end)
else:
    other = 6 - start - end
    hanoi(n-1, start, other)
    hanoi(1, start, end)
    hanoi(n-1, other, end)
```

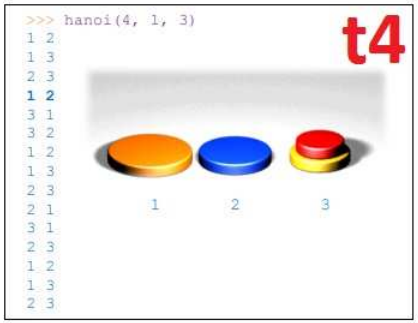


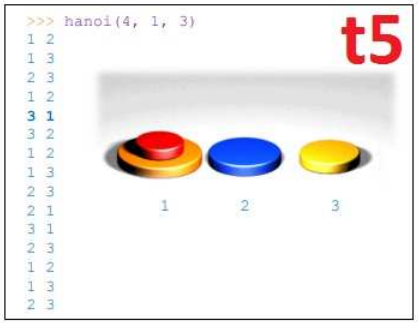

t0
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

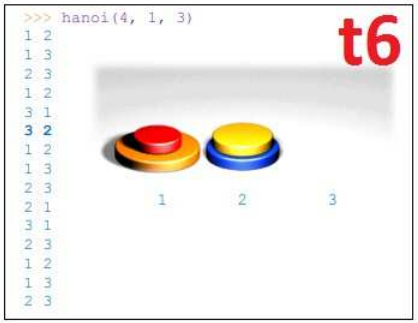

t1
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

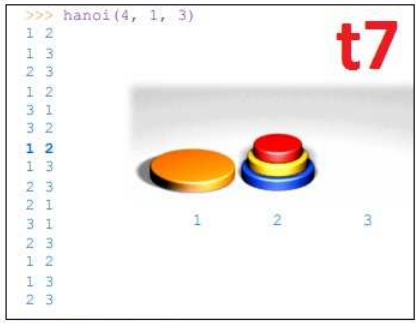

t2
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

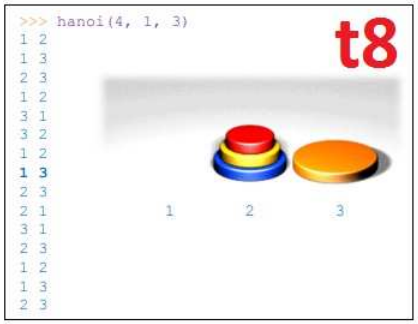

t3
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

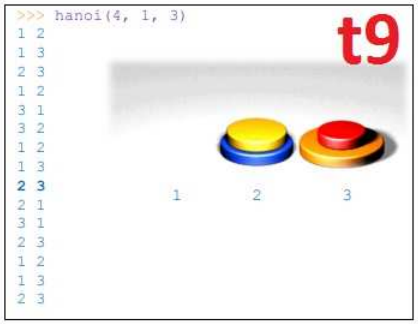

t4
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

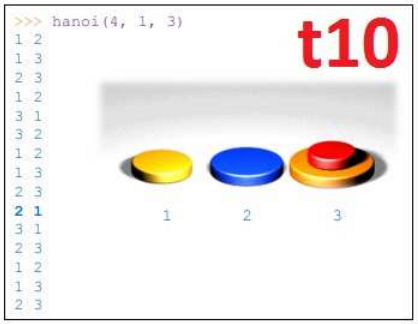

t5
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

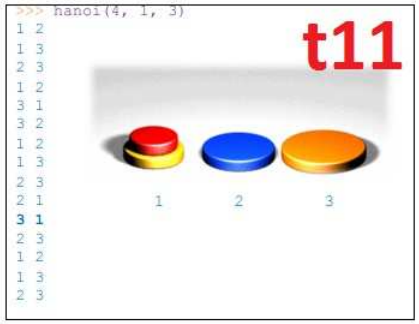

t6
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

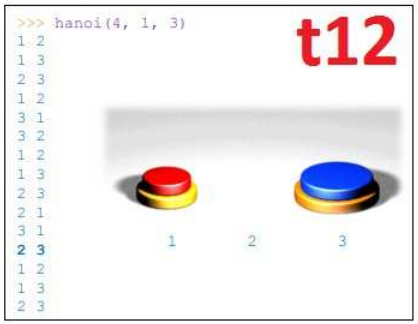

t7
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

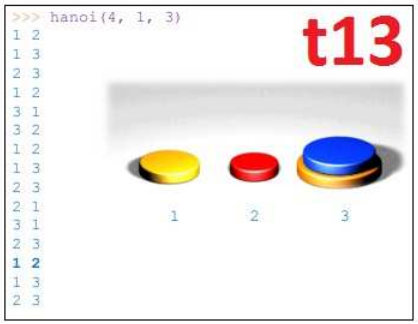

t8
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

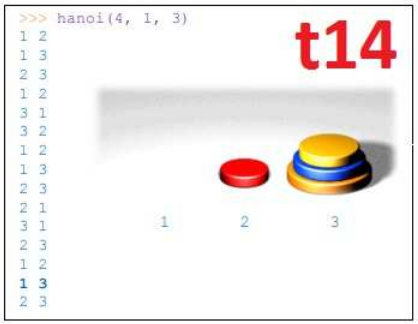

t9
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3

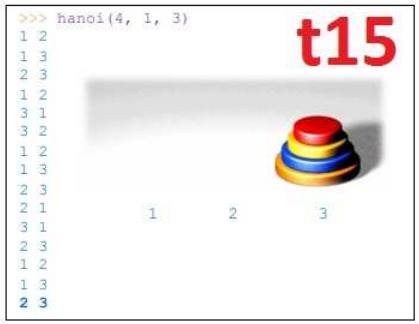

t10
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3


t11
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3


t12
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3


t13
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3


t14
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3


t15
>>> hanoi(4, 1, 3)
1 2
1 3
2 3
1 2
3 1
3 2
1 2
1 3
2 3
2 1
3 1
2 3
1 2
1 3
2 3
1 2 3